■ SRE RAG Agent v3.4 — Guia de Execução Local

Passo a Passo (Step by Step) para Inicialização, Operação e Validação do Laboratório

Este documento reúne as 3 formas homologadas de subir e operar o ambiente do **SRE Incident Simulation Lab v3.4** em ambiente local (WSL2 / Ubuntu ou Linux). Escolha a opção mais adequada ao seu objetivo de estudo ou teste.

■■ **Novidades & Hardening da v3.4:** • **Pydantic V2 ConfigDict:** Zero DeprecationWarnings no bootstrap de configs.
• **Health Check TTL Cache (5s):** Prevenção contra connection storms e medição de latência real de DB/Redis.
• **Async Embeddings:** *asyncio.to_thread* na inferência vetorial, liberando o event loop do FastAPI.
• **GZip & Nginx Cache:** Compressão transparente >1KB e cache estático (expires 1h) para assets imutáveis.
• **Defesa em Profundidade:** Security Headers automáticos (CSP, HSTS, XFO) e Rate Limiter (30 req/min no RAG).

Opção 1: Modo Desenvolvimento Local Rápido [MAIS RÁPIDO]

Ideal para desenvolvimento, testes de UI, simulação de incidentes e uso do War Room sem necessidade de inicializar contêineres Docker.

Passo 1: Navegue até o diretório raiz do projeto:

```
cd /home/magno/sre_rag_agent
```

Passo 2: Crie e ative o ambiente virtual Python 3.12 (caso ainda não exista):

```
python3 -m venv app/.venv
source app/.venv/bin/activate
```

Passo 3: Instale as dependências (com PyTorch CPU otimizado, sem CUDA bloat):

```
pip install --upgrade pip
pip install torch --index-url https://download.pytorch.org/whl/cpu
pip install -r app/requirements.txt
```

Passo 4: Inicie o servidor ASGI (Uvicorn) na porta 8080:

```
cd app
../app/.venv/bin/uvicorn main:app --host 0.0.0.0 --port 8080 --reload
# Ou execute diretamente na raiz: make dev
```

Passo 5: Acesse a interface e endpoints no navegador:

Dashboard Principal & Quiz	http://localhost:8080/dashboard	Simulador de incidentes, vitrine e runbooks em 5 fases
War Room (Terminal Interativo)	http://localhost:8080/dashboard#warroom	Terminal CLI seguro com comandos simulados (kubectl/helm)
Conceitos SRE & Arquitetura	http://localhost:8080/dashboard#concepts	Hub pedagógico de baixo nível (cgroups, CFS, epoll, probes)
Métricas & Histórico	http://localhost:8080/dashboard#history	Histórico de simulações, acertos e métricas de desempenho
Health Check da Aplicação	http://localhost:8080/healthz	Health check com cache TTL 5s e latência real do banco (v3.4)
Métricas Prometheus (RED)	http://localhost:8080/metrics	Métricas de Rate, Errors e Duration prontas para scraping

Opção 2: Modo Docker Compose (Multi-Network Isolado) [STACK SEGURA]

Sobe o ecossistema completo em contêineres: Nginx (Proxy reverso público na porta 80 com compressão GZip e cache), FastAPI (API), PostgreSQL e Redis interligados pela rede interna isolada backend_net.

Passo 1: Execute o build e inicialize os serviços em background:

```
cd /home/magno/sre_rag_agent
docker compose up -d --build
# Ou use o atalho homologado: make compose-up
```

Passo 2: Verifique a saúde e o status de inicialização dos contêineres:

```
docker compose ps
```

Passo 3: Acesso aos serviços via Nginx Reverse Proxy (Porta 80):

Interface Web Principal	http://localhost/dashboard	Porta 80 padrão com GZip ativo
Métricas Prometheus	http://localhost/metrics	Coleta RED para scrape
Health Check Nginx/API	http://localhost/healthz	Status com cache defensivo

Passo 4: Encerrar a stack quando finalizar os testes:

```
docker compose down
# Ou use o atalho homologado: make compose-down
```

Opção 3: Modo Cluster Kubernetes (Kind + Helm) [PRODUÇÃO REAL]

Simulação completa de cluster K8s local com orquestração Helm (Chart v0.2.0, appVersion 3.4), Prometheus Operator, Alertmanager e dashboards Grafana pré-configurados.

Passo 1: Provisionar o cluster Kind e a stack de monitoramento:

```
cd /home/magno/sre_rag_agent
make setup
```

Passo 2: Realizar o deploy do SRE RAG Agent via Helm (v3.4):

```
make deploy
```

Passo 3: Iniciar o encaminhamento de portas (Port-Forward):

```
make port-forward
```

Passo 4: Painéis e Credenciais de Acesso ao Cluster:

Aplicação SRE RAG	http://localhost:8080/dashboard	Sem autenticação (Ambiente de Lab)
Prometheus	http://localhost:9090	Métricas PromQL, SLIs e Recording Rules
Grafana	http://localhost:3000	**admin** / **admin** (Dashboards SRE)
Alertmanager	http://localhost:9093	Alertas ativos, rotas e inibições

Passo 5: Desligar e limpar todos os recursos do cluster:

```
make clean
```

Validação Rápida da Suíte de Testes & Qualidade (v3.4) [QUALIDADE & ACEITE]

Rotina completa de validação contínua para certificar que todos os 16 cenários de produção, o motor RAG semântico, os runbooks estruturados em 5 fases e as novas proteções técnicas da v3.4 estão 100% íntegros e operacionais.

Passo 1: Script de Validação Automatizada da v3.4 (12 Critérios de Aceite):

```
bash scripts/verify_v3.4.sh
# Executa a verificação dos 12 critérios de aceite: sintaxe, Pydantic V2 ConfigDict,
# TTL cache, async embeddings, gzip, security headers, rate limit e testes unitários.
```

Passo 2: Executar a suíte de testes unitários e de integração:

```
make test
# Executa: PYTHONPATH=.:app ./app/.venv/bin/pytest tests/ -v
```

Módulos e Escopo de Testes Automatizados Cobertos:

Módulo de Teste	Arquivo	O que é validado
Cenários de Incidente	tests/test_incidents.py	16 cenários de produção, integridade dos runbooks em 5 fases, balanceamento em 4 dificuldades (Fácil, Médio, Difícil, Extremo) e histórico de zero-repetição.
Chaos Monkey 2.0	tests/test_chaos.py	Injeção controlada de falhas HTTP 500, auditoria em tempo real, geração de eventos de incidentes e exportação de métricas RED.
Motor RAG Semântico	tests/test_rag.py	Indexação vetorial de runbooks, inferência com SentenceTransformer (asyncio.to_thread), cálculo de similaridade e acurácia de recuperação.

Passo 3: Executar a avaliação de acurácia com o benchmark SRE-Eval:

```
make eval
# Executa: PYTHONPATH=.:app ./app/.venv/bin/python tests/eval/evaluator.py
# Gera o relatório de acurácia, relevância semântica e latência de resposta da IA.
```

■ Dicas de Observabilidade & Troubleshooting Rápido no Lab:

- **Status dos SLOs em Tempo Real:** Execute make slo-status para verificar a taxa de queima (Burn Rate) de Error Budget instantânea.
- **Simulação de Carga Contínua:** Execute make load-test para disparar requisições concorrentes e testar o HPA e o Rate Limiter.
- **Auditoria do Chaos Monkey:** Monitore o terminal ou os logs com docker compose logs -f api para inspecionar falhas injetadas.
- **Verificação de Health Cache:** Execute requisições sucessivas a /healthz para constatar a resposta sub-milissegundo via cache TTL.